

Troubleshooting Guide

Common issues and solutions for the Gema Backend API.

Common Issues

Authentication & Authorization

Issue: "Invalid or expired token"

Symptoms: 401 Unauthorized responses on protected endpoints

Solutions:

```
# Check if token is properly formatted
curl -X GET http://localhost:5000/api/auth/me \
  -H "Authorization: Bearer your_token_here" \
  -v

# Verify token hasn't expired - refresh if needed
curl -X POST http://localhost:5000/api/auth/refresh-token \
  -H "Content-Type: application/json" \
  -d '{"refreshToken":"your_refresh_token"}'
```

Common Causes:

- Token expired (access tokens last 7 days)
- Malformed Authorization header (missing "Bearer " prefix)
- Invalid JWT secret in environment variables
- Token generated with different secret

Issue: "User not found" after login

Symptoms: Login succeeds but subsequent API calls fail

Solutions:

1. Check if user exists in database:

```
# MongoDB query
db.users.findOne({email: "user@example.com"})
```

2. Verify JWT payload:

```
// Decode token to check user ID
const jwt = require('jsonwebtoken');
const decoded = jwt.decode(token);
console.log('User ID in token:', decoded.id);
```

Issue: "Insufficient permissions" (403 Forbidden)

Symptoms: User authenticated but can't access certain endpoints

Solutions:

1. Check user role:

```
curl -X GET http://localhost:5000/api/auth/me \
  -H "Authorization: Bearer your_token"
# Look for "role" field in response
```

2. Verify endpoint requirements:

- Customer: Can access orders, tickets, reviews
- Vendor: Can manage events, view analytics
- Admin: Full system access

Database Connection

Issue: "MongoNetworkError" or connection timeout

Symptoms: Cannot connect to MongoDB

Solutions:

1. **Local MongoDB:**

```
# Check if MongoDB is running
sudo systemctl status mongod

# Start MongoDB service
sudo systemctl start mongod

# Check connection string
echo $MONGODB_URI
```

2. **MongoDB Atlas:**

```
# Test connection string
curl -X GET http://localhost:5000/api/health
# Check database status in response
```

Common Causes:

- MongoDB service not running locally
- Incorrect connection string in .env
- Network firewall blocking connection
- MongoDB Atlas IP whitelist restrictions
- Wrong database name in connection string

Issue: Validation errors on database operations

Symptoms: "ValidationError" or schema-related errors

Solutions:

1. Check required fields:

```
// Example for User model
const userData = {
  firstName: 'John',      // Required
  lastName: 'Doe',        // Required
  email: 'john@test.com', // Required, unique
  role: 'customer'        // Required
};
```

2. Verify data types match schema

3. Check for duplicate keys (email, username)

Environment Configuration

Issue: Application fails to start

Symptoms: Server crashes on startup or missing environment variables

Solutions:

1. **Check environment variables:**

```
# List all required variables
cat .env.example

# Compare with your .env file
cat .env
```

2. **Required variables checklist:**

```
✓ NODE_ENV=development
✓ PORT=5000
✓ MONGODB_URI=mongodb://localhost:27017/gema
✓ JWT_SECRET=your_jwt_secret_32_chars_minimum
✓ JWT_REFRESH_SECRET=your_refresh_secret
✓ FIREBASE_PROJECT_ID=your-project-id
✓ FIREBASE_PRIVATE_KEY="-----BEGIN PRIVATE KEY-----..."
✓ FIREBASE_CLIENT_EMAIL=firebase-adminsdk@...
```

3. Firebase key formatting:

```
# Correct format (with literal \n characters)
FIREBASE_PRIVATE_KEY="-----BEGIN PRIVATE KEY-----\nMIIEvQIB...your_key_here...\n-----END PRIVATE KEY-----\n"
```

Issue: CORS errors in browser

Symptoms: "Access-Control-Allow-Origin" errors

Solutions:

1. Check FRONTEND_URL in .env:

```
FRONTEND_URL=http://localhost:3000
# or
FRONTEND_URL=https://your-frontend-domain.com
```

2. Verify CORS configuration:

```
# Test preflight request
curl -X OPTIONS http://localhost:5000/api/events \
  -H "Origin: http://localhost:3000" \
  -H "Access-Control-Request-Method: GET"
```

File Upload Issues

Issue: File upload fails or returns errors

Symptoms: Upload endpoints returning 400 or 500 errors

Solutions:

1. Check file size limits:

- Single files: Max 5MB
- Multiple files: Max 5 files, 25MB total
- Event images: Max 10 files

2. Verify file types:

```
// Allowed types
const allowedTypes = {
  images: ['image/jpeg', 'image/png', 'image/gif'],
  documents: ['application/pdf', 'application/msword']
};
```

3. Test upload with curl:

```
curl -X POST http://localhost:5000/api/uploads/single \
  -H "Authorization: Bearer your_token" \
  -F "file=@path/to/your/image.jpg"
```

Issue: Cloudinary upload failures

Symptoms: Files upload locally but fail in production

Solutions:

1. Check Cloudinary credentials:

```
CLOUDINARY_CLOUD_NAME=your_cloud_name
CLOUDINARY_API_KEY=your_api_key
CLOUDINARY_API_SECRET=your_api_secret
CLOUDINARY_SECURE=true
```

2. Test Cloudinary connection:

```
curl -X GET http://localhost:5000/api/health
# Look for cloudinary service status
```

Payment Processing

Issue: Stripe payment failures

Symptoms: Payment intent creation fails or webhooks not working

Solutions:

1. Check Stripe keys:

```
STRIPE_SECRET_KEY=sk_test_... # or sk_live_...
STRIPE_PUBLISHABLE_KEY=pk_test_... # or pk_live_...
STRIPE_WEBHOOK_SECRET=whsec_...
```

2. Test webhook endpoint:

```
# Webhook should be accessible
curl -X POST http://localhost:5000/api/payments/webhook \
-H "stripe-signature: test"
```

3. Verify webhook URL in Stripe Dashboard:

- Development: <https://your-ngrok-url.ngrok.io/api/payments/webhook>
- Production: <https://your-domain.com/api/payments/webhook>

Email Service Issues

Issue: Email verification not sending

Symptoms: Users not receiving OTP emails

Solutions:

1. Check email configuration:

```
EMAIL_HOST=smtp.gmail.com
EMAIL_PORT=587
EMAIL_USERNAME=your-email@gmail.com
EMAIL_PASSWORD=your-16-char-app-password
EMAIL_FROM=noreply@your-domain.com
```

2. Gmail App Password setup:

- Enable 2-factor authentication
- Generate app-specific password
- Use app password (not regular password)

3. Test email sending:

```
curl -X POST http://localhost:5000/api/auth/resend-verification-email \
-H "Content-Type: application/json" \
-d '{"email": "test@example.com"}'
```

Performance Issues

Issue: Slow API responses

Symptoms: Requests taking longer than expected

Solutions:

1. Check database queries:

```
# Enable MongoDB query profiling
db.setProfilingLevel(2)
db.system.profile.find().limit(5).sort({ts: -1}).pretty()
```

2. Add database indexes:

```
// Common indexes
db.events.createIndex({category: 1, city: 1})
db.users.createIndex({email: 1})
db.orders.createIndex({user: 1, createdAt: -1})
```

3. Monitor memory usage:

```
# Check Node.js memory
curl -X GET http://localhost:5000/api/health
# Look for memory usage stats
```

Debugging Tools

Enable Debug Logging

```
# Development environment
DEBUG=app:* npm run dev

# Specific modules
DEBUG=app:auth,app:db npm run dev
```

Health Check Endpoint

```
# Comprehensive system health
curl -X GET http://localhost:5000/api/health

# Expected response
{
  "success": true,
  "data": {
    "status": "healthy",
    "database": {"status": "connected"},
    "services": {
      "firebase": "connected",
      "stripe": "connected",
      "cloudinary": "connected"
    }
  }
}
```

Log File Locations

```
# Application logs
tail -f logs/app.log

# Error logs
tail -f logs/error.log

# Access logs
tail -f logs/access.log
```

Development Tools

Database GUI Tools

- **MongoDB Compass:** Visual database management
- **Studio 3T:** Advanced MongoDB client
- **Robo 3T:** Lightweight MongoDB GUI

API Testing Tools

- **Postman:** Full-featured API testing
- **Insomnia:** Clean API client
- **curl:** Command-line testing
- **Thunder Client:** VS Code extension

Monitoring Tools

- **Render Dashboard:** Production monitoring
- **MongoDB Atlas:** Database monitoring
- **Stripe Dashboard:** Payment monitoring
- **Cloudinary Dashboard:** Media monitoring

Getting Help

Log Analysis

When reporting issues, include:

1. Error logs:

```
# Application logs
npm run dev 2>&1 | tee debug.log
```

2. Environment info:

```
node --version
npm --version
cat .env.example # Don't share actual .env
```

3. Request details:

```
curl -X POST http://localhost:5000/api/endpoint \
  -H "Content-Type: application/json" \
  -d '{"data": "here"}' \
  -v # Verbose output
```

Common Commands for Debugging

```
# Reset local development
npm run build
rm -rf node_modules package-lock.json
npm install
npm run db:seed

# Check environment
env | grep -E "(MONGO|JWT|FIREBASE|STRIPE)"

# Test all services
curl -X GET http://localhost:5000/api/health

# Database connection test
node -e "
  require('mongoose').connect(process.env.MONGODB_URI)
    .then(() => console.log('DB Connected'))
    .catch(err => console.error('DB Error:', err))
"
```

Support Channels

- **Documentation:** Check [Master Index \(../README.md\)](#)
- **Issues:** GitHub Issues for bug reports
- **Discussions:** GitHub Discussions for questions

Related Resources:

- [Getting Started → \(../architecture/getting-started.md\)](#) - Initial setup guide
- [Deployment → \(../deployment/deployment-guide.md\)](#) - Production deployment
- [API Reference → \(../api/api-reference.md\)](#) - Complete API documentation

